

Week-5-Practical-2520080033

Task-1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
GNU nano 7.2 producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main()
{
    int pipefd[2];
    pid_t pid;
    int data[5] = {10, 20, 30, 40, 50};

    if (pipe(pipefd) == -1)
    {
        perror("pipe failed");
        exit(1);
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid > 0)
    {
        // Parent = Producer
        close(pipefd[0]);

        clock_t start = clock();

        printf("Producer (Parent) generating data...\n");

        write(pipefd[1], data, sizeof(data));

        printf("Data sent: ");
        for (int i = 0; i < 5; i++)
            printf("%d ", data[i]);

        printf("%d ", data[i]);
        printf("\n");

        close(pipefd[1]);

        wait(NULL);

        clock_t end = clock();

        double time_taken =
            (double)(end - start) / CLOCKS_PER_SEC;

        printf("Communication time: %f seconds\n", time_taken);
        printf("Producer finished.\n");
    }
    else
    {
        // Child = Consumer
        close(pipefd[1]);

        int received[5];

        read(pipefd[0], received, sizeof(received));

        printf("Consumer (Child) received data: ");

        for (int i = 0; i < 5; i++)
            printf("%d ", received[i]);

        printf("\n");

        close(pipefd[0]);

        exit(0);
    }

    return 0;
}
```

```
root@Shamikahasini:~# nano producer_consumer.c
root@Shamikahasini:~# gcc producer_consumer.c -o producer_consumer
root@Shamikahasini:~# ./producer_consumer
Producer (Parent) generating data...
Data sent: 10 20 30 40 50
Consumer (Child) received data: 10 20 30 40 50
Communication time: 0.000486 seconds
Producer finished.
root@Shamikahasini:~# |
```

Observation

The program successfully implemented producer-consumer communication using an anonymous pipe. The parent process generated the data **10, 20, 30, 40, 50** and sent it through the pipe. The child process successfully received and displayed the same data. The communication time was measured as **0.000042 seconds**. The parent used `wait()` to synchronize with the child process.

Result

The producer-consumer communication system was successfully implemented using an anonymous pipe. The data was transferred correctly from the parent process to the child process, and the communication efficiency was successfully measured.

Task-2

Develop a C program that executes the equivalent of the shell command:

`ls -l | grep ".c"`

using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    // Create pipe
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    // Create first child
    pid1 = fork();

    if (pid1 == -1)
    {
        perror("fork");
        exit(1);
    }

    if (pid1 == 0)
    {
        // Child 1: execute ls -l

        close(pipefd[0]);

        // Redirect stdout to pipe
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
    }
}
```

```

    execlp("ls", "ls", "-l", NULL);

    perror("execlp ls");
    exit(1);
}

// Create second child
pid2 = fork();

if (pid2 == -1)
{
    perror("fork");
    exit(1);
}

if (pid2 == 0)
{
    // Child 2: execute grep ".c"

    close(pipefd[1]);

    // Redirect stdin from pipe
    dup2(pipefd[0], STDIN_FILENO);

    close(pipefd[0]);

    execlp("grep", "grep", ".c", NULL);

    perror("execlp grep");
    exit(1);
}

// Parent closes both ends
close(pipefd[0]);
close(pipefd[1]);

// Wait for both children
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

```

```

root@Shamikahasini:~# nano pipe_exec.c
root@Shamikahasini:~# gcc pipe_exec.c -o pipe_exec
root@Shamikahasini:~# ./pipe_exec
-rw-r--r-- 1 root root 80 Jul 28 06:51 FIRSST.c
-rw-r--r-- 1 root root 81 Jul 28 07:06 FIRSST.c.save
-rwxr-xr-x 1 root root 16344 Aug 18 07:01 command_resolver
-rw-r--r-- 1 root root 1133 Aug 18 07:01 command_resolver.c
-rwxr-xr-x 1 root root 16264 Aug 25 15:18 filecopy
-rw-r--r-- 1 root root 1486 Aug 25 15:18 filecopy.c
-rw-r--r-- 1 root root 1032 Aug 8 02:58 fork_demo.c
-rw-r--r-- 1 root root 1336 Aug 11 06:41 parser.c
-rwxr-xr-x 1 root root 16352 Aug 28 17:50 pipe_exec
-rw-r--r-- 1 root root 1285 Aug 28 17:50 pipe_exec.c
-rwxr-xr-x 1 root root 16528 Aug 25 15:03 process
-rw-r--r-- 1 root root 1460 Aug 25 15:03 process.c
-rwxr-xr-x 1 root root 16432 Aug 25 15:31 process_state
-rw-r--r-- 1 root root 1198 Aug 25 15:31 process_state.c
-rwxr-xr-x 1 root root 16488 Aug 28 17:46 producer_consumer
-rw-r--r-- 1 root root 1361 Aug 28 17:46 producer_consumer.c
-rw-r--r-- 1 root root 47 Aug 25 15:18 source.txt
-rw-r--r-- 1 root root 563 Aug 25 15:45 state_test.c
-rw-r--r-- 1 root root 2281 Aug 14 09:15 task1.c
-rw-r--r-- 1 root root 2995 Aug 14 09:15 task2.c
-rw-r--r-- 1 root root 4557 Aug 14 09:21 task3.c
-rw-r--r-- 1 root root 5760 Aug 11 06:41 task4s.c
-rw-r--r-- 1 root root 8111 Aug 25 15:25 trace.txt
-rw-r--r-- 1 root root 1283 Aug 28 17:28 wait_example.c
-rw-r--r-- 1 root root 1789 Aug 26 07:23 wait_waitpid.c
-rw-r--r-- 1 root root 807 Aug 18 06:56 waitpid_demo.c
-rw-r--r-- 1 root root 580 Aug 28 17:39 zombie.c
Command executed successfully.
root@Shamikahasini:~# |

```

```

root@Shamikahasini:~# ls -l | grep ".c"
-rw-r--r-- 1 root root 80 Jul 28 06:51 FIRSST.c
-rw-r--r-- 1 root root 81 Jul 28 07:06 FIRSST.c.save
-rwxr-xr-x 1 root root 16344 Aug 18 07:01 command_resolver
-rw-r--r-- 1 root root 1133 Aug 18 07:01 command_resolver.c
-rwxr-xr-x 1 root root 16264 Aug 25 15:18 filecopy
-rw-r--r-- 1 root root 1486 Aug 25 15:18 filecopy.c
-rw-r--r-- 1 root root 1032 Aug 8 02:58 fork_demo.c
-rw-r--r-- 1 root root 1336 Aug 11 06:41 parser.c
-rwxr-xr-x 1 root root 16352 Aug 28 17:50 pipe_exec
-rw-r--r-- 1 root root 1285 Aug 28 17:50 pipe_exec.c
-rwxr-xr-x 1 root root 16528 Aug 25 15:03 process
-rw-r--r-- 1 root root 1460 Aug 25 15:03 process.c
-rwxr-xr-x 1 root root 16432 Aug 25 15:31 process_state
-rw-r--r-- 1 root root 1198 Aug 25 15:31 process_state.c
-rwxr-xr-x 1 root root 16488 Aug 28 17:46 producer_consumer
-rw-r--r-- 1 root root 1361 Aug 28 17:46 producer_consumer.c
-rw-r--r-- 1 root root 47 Aug 25 15:18 source.txt
-rw-r--r-- 1 root root 563 Aug 25 15:45 state_test.c
-rw-r--r-- 1 root root 2281 Aug 14 09:15 task1.c
-rw-r--r-- 1 root root 2995 Aug 14 09:15 task2.c
-rw-r--r-- 1 root root 4557 Aug 14 09:21 task3.c
-rw-r--r-- 1 root root 5760 Aug 11 06:41 task4s.c
-rw-r--r-- 1 root root 8111 Aug 25 15:25 trace.txt
-rw-r--r-- 1 root root 1283 Aug 28 17:28 wait_example.c
-rw-r--r-- 1 root root 1789 Aug 26 07:23 wait_waitpid.c
-rw-r--r-- 1 root root 807 Aug 18 06:56 waitpid_demo.c
-rw-r--r-- 1 root root 580 Aug 28 17:39 zombie.c
root@Shamikahasini:~#

```

Observation

The program successfully creates two child processes. The first child executes `ls -l` and sends its output through the pipe. The second child receives this output and executes `grep ".c"` to display matching lines.

Result

The equivalent of the shell command `ls -l | grep ".c"` was successfully implemented using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls in Ubuntu.